

Introducción – Criterios de Evaluación - Paradigmas

Lenguajes

Un lenguaje en general, es el medio del que se vale cualquier individuo para comunicar ideas y experiencias a otros individuos. Estas ideas y experiencias están “almacenadas” de alguna manera el individuo emisor y “almacenadas” tal vez con otra representación en el receptor.

El **lenguaje de programación** en general tiene la misma idea, solo que el receptor es una computadora, por ello es una de las principales herramientas en el proceso de desarrollo del software.

El arte de diseñar lenguajes

Hay una gran variedad de lenguajes y continúan apareciendo nuevos. ¿Por qué?

- Evolución
- Propósito especial
- Preferencia personal

¿Qué convierte a un lenguaje en **exitoso**?

- Poder expresivo
- Facilidad de uso para novatos (baja curva de aprendizaje)
- Facilidad de implementación
- Estandarización
- Código abierto
- Excelentes compiladores
- Inercia, economía, quien lo promueve

Introducción a lenguajes de programación

Originalmente las computadoras eran grandes y costosas, eran programadas en lenguajes máquina (expresado como números en hexadecimal).

A medida que se comenzaron a escribir programas más largos, se volvió claro que era necesario utilizar un lenguaje menos propenso a errores (notación con abreviaturas nemotécnicas – lenguaje ensamblador). La traducción quedó a cargo del ensamblador.

Con la evolución de las computadoras y el desarrollo de diseños se volvió frustrante reescribir los programas para cada nueva máquina. También se volvió humanamente complicado entender los programas en lenguaje ensamblador.

Se hizo necesario contar con lenguajes independientes de la máquina.

Lenguajes

Hay diferentes taxonomías para los lenguajes:

- Generaciones (primera, segunda, tercera, cuarta, quinta)
- Paradigma
- Dominio de aplicación
- Tipo de traductor

Otra Clasificación

Declarativa:

Funcional	Lisp/Scheme, ML, Haskell
Dataflow (flujo de información)	Id, Val
Lógica, basada en restricciones	Prolog, aspectos programables de las hojas de cálculo (Excel, Lotus, etc.)
Basada en templates	XSLT

Imperativa:

Von Newmann	C, Ada, Fortran,
Scripting	Perl, Python, PHP,
Orientada a objetos	Smalltalk, Eiffel, C++, Java

Desarrollo de Lenguajes de Programación

El **lenguaje de programación** surge de la vinculación entre las diferentes metodologías de diseño y las arquitecturas de la computadoras.

Otro de los factores que terminan teniendo especial relevancia es el dominio de aplicación pretendido.

Características deseables en el software

De cualquier software se espera, básicamente, que sea:

- confiable
- extensible
- reusable
- robusto
- eficiente
- flexibilidad
- etc.

Características deseables en el software

Los **lenguajes de programación** también son software pero al tener otro tipo de propósito se espera que posea otras características:

- facilidad de lectura y escritura (legibilidad)
- seguridad
- costo

Readability (facilidad de lectura)

La facilidad para leer e interpretar programas es fundamental.

Características que contribuyen a la legibilidad:

- **Simplicidad**
 - Se obtiene en gran medida de combinar un número de pequeño de constructores primitivos y un uso limitado (ni mucho-ni poco) del concepto de ortogonalidad.
- **Ortogonalidad**
 - Básicamente significa que un conjunto pequeño de constructores primitivos, puede ser combinado en número relativamente pequeño a la hora de construir estructuras de control y datos. Cada combinación es legal y con sentido.
- **Estructuras de control y de datos**
 - Contar con estructuras legibles (sólo uso de goto – agregado del tipo boolean)
- **Consideraciones sobre la sintaxis**
 - Formas de los identificadores – palabras reservadas – forma y significado (por ejemplo el modificador static de JAVA)

Writability (facilidad de escritura)

La facilidad para crear programas de un dominio. Este criterio depende del dominio de aplicación del lenguaje.

- **Simplicidad y Ortogonalidad**
 - Mucha ortogonalidad genera una menor detección de errores. Si el lenguaje no es simple solo se aprende y utiliza una pequeña porción de él.
- **Soporte para la abstracción**
 - Capacidad de definir y usar estructuras u operaciones complicadas de manera que sea posible ignorar muchos de los detalles.
- **Expresividad**
 - El lenguaje posee formas relativamente convenientes de expresar ciertas operaciones, por ejemplo `contador++` en lugar de `contador = contador + 1` o el uso del `for` en lugar del `while` equivalente.

Reliability (fiabilidad)

- **Chequeo de tipos**
 - Cuanto antes se encuentren errores menos costoso resulta realizar los arreglos que se requieran.
- **Manejo de excepciones**
 - La habilidad para interceptar errores en tiempo de ejecución, tomar medidas correctivas y continuar.

Costo

- **Aprender** (a escribir programas cercanos a aplicaciones particulares)
- **Usar** (entrenar programadores para el uso del lenguaje)
- **Compilar**
- **Ejecutar**
- **Sistema de implementación del lenguaje** (disponibilidad de compiladores gratuitos)
- **Confiabilidad** (escasa confiabilidad conlleva altos costos)
- **Programas de mantenimiento**

Paradigmas

Lenguajes y Paradigmas

Metodología de diseño

es un conjunto de métodos y pautas que guían el proceso de desarrollo.

Paradigma de Programación

- Conjunto coherente de métodos para resolver un problema
- Tiene principio básico
- Guía el proceso de desarrollo del software

Colección de patrones conceptuales integrados que orientan el proceso de desarrollo de software y determinan la estructura de un programa válido.

Lenguajes y Paradigmas: “soportar” o “admitir”

Un lenguaje “soporta” un paradigma si provee mecanismos que facilitan su implementación eficiente. Es decir, implementa mecanismos que permiten la sencilla aplicación del paradigma (impone restricciones para respetarlo).

Un lenguaje “admite” un paradigma si es posible escribir programas siguiendo los lineamientos del paradigma, pero hacerlo demanda un esfuerzo notable.

El lenguaje no exige la aplicación del paradigma, permite programar de acuerdo al paradigma pero sin proveer facilidades.

Lenguajes y Paradigmas: ¿focalizando en acciones o datos?

Diseño top-down
Programación
estructurada
Programación modular

Algoritmos

Abstracción de datos

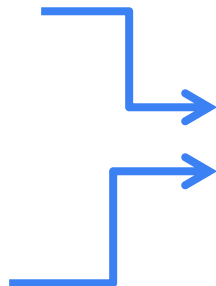
Datos

Clasificación

Algoritmos
+
Datos

Paradigma
Imperativo

Paradigma Orientación
a objetos



Paradigma Imperativo

Un programa es una secuencia de instrucciones que indican el flujo de la ejecución.

Señal dada para que se realice el cambio de estado del autómata

Las principales características son:

1. Ejecución secuencial de instrucciones
2. Uso de variables representando valores de locaciones de memoria
3. Uso de sentencias de asignación para cambiar los valores de las variables, permitiendo así al programa operar sobre las locaciones de memoria

Paradigma Imperativo

Se busca estructurar el control realizando una programación estructurada y modular con abstracción de datos para fomentar la reusabilidad y extendibilidad.

Los programas se construyen siguiendo una aproximación:

- Top-down

- Modular

- Solo sub-programas
- Dividir para conquistar
- Existen abstracciones algorítmicas

Abstracción a nivel instrucción
Agrupar instrucciones en unidades –
Procedure de Pascal

Abstracción de expresiones
Function de Pascal

Paradigma Imperativo

Los programas se construyen siguiendo una aproximación: **Modular**

- Programación Estructurada

- Estructuras de control a nivel instrucción
- Inhibición del uso de estructuras de control

- Programación Modular



Módulo

con

Independencia de significado
Independencia de implementación

- Abstracción de datos

- Reconocer entidades abstractas
(hallar representación para los datos)
- Reconocer operaciones lógicas
(transformar en operaciones concretas)

Paradigma Imperativo - Limitaciones

- Dificultad para razonar
 - Alta dependencia de la arquitectura
 - una variable es el de abstracción de celda de memoria
- Transparencia referencial
 - Asignaciones
- Efectos colaterales y aliasing
- Accesibilidad vulnerable
- Estructuras de control: desdibujan la lógica del programa generando problemas de mantenimiento y seguridad

Paradigma Orientado a Objetos

Se caracteriza por reconocer las entidades del problema (similar a la abstracción de datos)

- Entidad = **Objetos**
- Comunicación por **mensajes**, diferente a la semántica de llamadas a procedimiento

Caracterizado por atributos y comportamiento
(de acuerdo a su propósito y habilidades)

Objetivos:

- Mejorar la **reusabilidad** del software
- Mejorar la **extendibilidad** del software

Para algunos  **Quiebre total con lo anterior**
Nuevo eslabón de la línea estructurada

Paradigma Orientado a Objetos

Los lenguajes orientados a objetos deben proveer:

- Comunicación por mensajes
- Concepto de objeto con estado interno y servicios
- Soportar clasificación

El mínimo para los diferentes autores es:

- Abstracción de datos + herencia
 - + encapsulamiento
 - + polimorfismo

Para nosotros la abstracción de datos y el encapsulamiento son conceptos que deben ir juntos.

Para algunos autores polimorfismo es caso particular de herencia y para otros polimorfismo, herencia y sobrecarga son conceptos diferentes

Otros autores requieren además ligadura dinámica y otros soporte de concurrencia (aunque esto último no es inherente al paradigma)

Paradigmas no convencionales

Las formalizaciones abstractas, como las máquinas de Turing, el cálculo lambda, etc., brindan formas de expresión y razonamiento que permiten obtener construcciones útiles y adecuadas para descubrir problemas

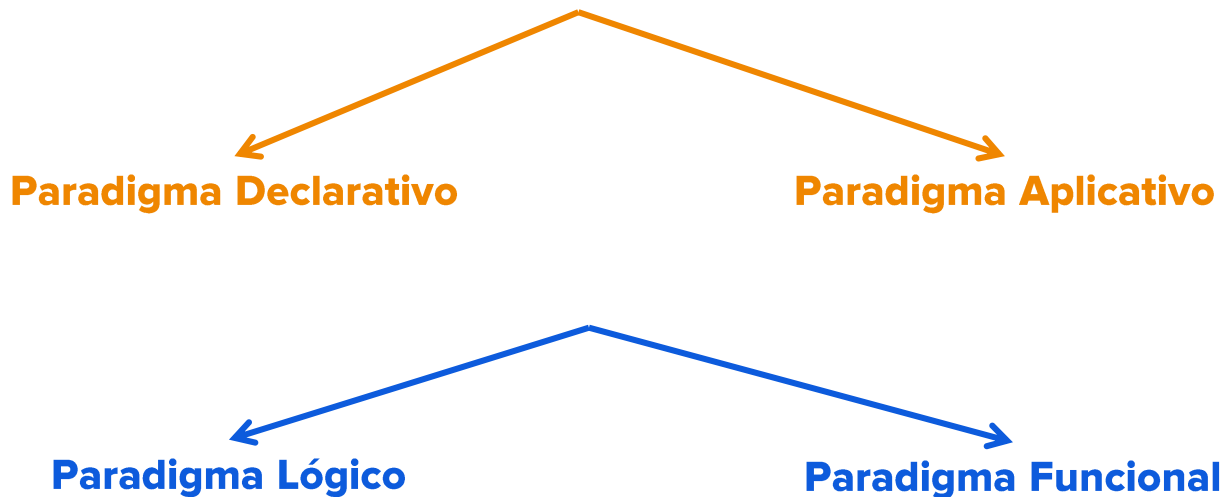
Redescubrir los formalismos como una forma resolver problemas, utilizándolos en el diseño y la implementación

Reciben el nombre de no convencionales porque su forma de especificar y ejecutar están “alejadas” de la arquitectura.

Paradigmas no convencionales

Alejados totalmente del control del paradigma imperativo.

La lógica y la matemática nos brindan formas de expresión como:



Paradigma Declarativo

Un programa establece un conjunto de propiedades que describen como alcanzar la solución. En ningún caso se indican cómo se computa

Un lenguaje declarativo, brinda las facilidades para especificar estas propiedades y una forma de computar que sea transparente

El principal problema es la eficiencia

La ventaja es que los programas que se obtienen son elegantes y concisos

Paradigma Lógico

Basado en lógica de primer orden (lenguaje preciso para expresar conocimiento)

- Lenguaje representativo: ProLog

Los fundamentos del paradigma son:

- a) Deducir consecuencias a partir de premisas
- b) Estudiar o decidir el valor de verdad de una sentencia a partir del valor de verdad de otras
- c) Establecer la consistencia entre hechos y verificar la validez de argumentos

Paradigma Lógico

Características de los lenguajes lógicos

- a) Eliminación del control
- b) El concepto de variable es más matemático, son nombres que retienen valores
- c) Establecen “que” es lo que se debe hacer sin dar ninguna especificación sobre el “como” hacerlo

Características de los programas lógicos

- a) Conjuntos de axiomas que establecen relaciones
- b) Definen un conjunto de consecuencias que determinan el significado
- c) Son teoremas y la ejecución es una prueba automática

Paradigma Aplicativo

Basado en el uso de funciones, de ahí que habitualmente se hable de él como paradigma funcional. Muy popular en la resolución de problemas de inteligencia artificial, matemática, lógica, procesamiento paralelo.

Algunas ventajas:

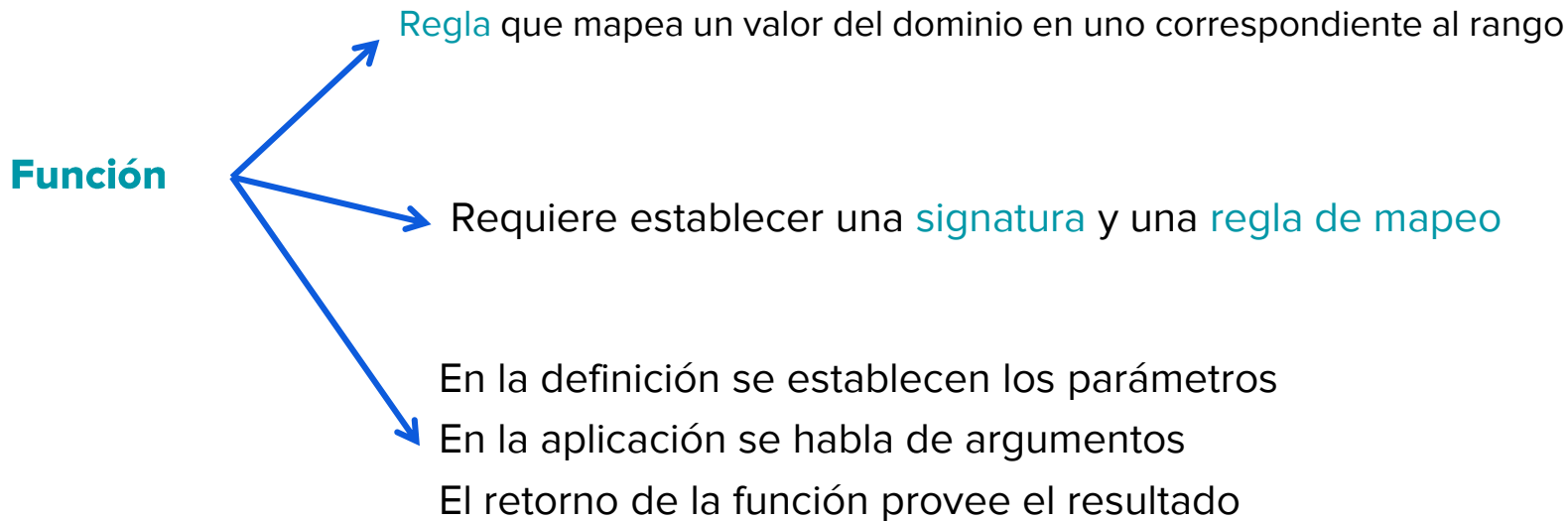
- Vista uniforme de programa y función
- Tratamiento de funciones como datos
- Liberación de efectos colaterales
- Manejo automático de memoria

Desventaja:

- Ineficiencia de ejecución

Paradigma Funcional

La esencia de esta metodología está en componer funciones para definir otras más complejas.



Paradigma Funcional

Características de los lenguajes funcionales

- a) Define un conjunto de datos
- b) Provee un conjunto de funciones primitivas
- c) Provee un conjunto de formas funcionales
- d) Requiere de un operador de aplicación

Características de los programas funcionales

- a) Semántica basada en valores
- b) Transparencia referencial
- c) Regla de mapeo basada en combinación o composición
- d) Las funciones son “ciudadanos” de primer orden

Integración de paradigmas

Aparecen los lenguajes híbridos, principalmente debido a necesidades heterogéneas y la interoperabilidad disponible.

Lenguajes Script

Motivación

La programación tradicional construye aplicaciones auto-contenidas. Actualmente se espera que las computadoras coordinen múltiples programas.

Un ejemplo se da en la creación de páginas web dinámicas, se requiere autenticación, autorización, acceso a base de datos, manipulación de imágenes, leer y escribir texto HTML.

Es posible escribir código de coordinación en lenguajes convencionales (Java, C, etc.) pero no siempre es fácil. Persigue eficiencia, mantenibilidad, portabilidad y detección estática de errores.

Lenguajes Script

Motivación

Los lenguajes convencionales tienden a mejorar eficiencia, mantenibilidad, portabilidad y detección estática de errores.

Los tipos se construyen alrededor de conceptos a nivel hardware como enteros de tamaño fijo, punto flotante, caracteres y arreglos.

Los lenguajes script tienden a mejorar flexibilidad, desarrollo rápido y chequeo dinámico. Su sistema de tipos se construye sobre conceptos de mas alto nivel como tablas, patrones, listas y archivos.

Lenguajes Script

Los lenguajes script de propósito general (Perl, Python) suelen conocerse como *glue-languages*.

Se diseñaron para “pegar” programas existentes a fin de construir un sistema mas grande.

Se utilizan como lenguajes de extensión, ya que permiten al usuario adaptar o extender las funcionalidad de las herramientas script.

¿Qué es un lenguaje Script?

Es difícil definirlos con precisión, aunque hay varias características que tienden a tener en común

Estos lenguajes tienen dos tipos de ancestros:

- Interpretes de líneas de comando o “shells”
- Herramientas para procesamiento de texto y generación de reportes.

“Los lenguajes script asumen la existencia de componentes útiles en otros lenguajes. Su intención no es escribir aplicaciones desde el comienzo sino por combinación de componentes” (John Ousterhout – creador de TCL)

Características comunes

- [Uso “batch” e interactivo](#): pocos lenguajes script usan compilación just-in-time. En general compilan o interpretan línea por línea.
- [Economía de expresión](#): soportan desarrollo rápido y uso interactivo. Tratan de evitar declaraciones extensas y las estructuras propias de los lenguajes convencionales.
- [Falta de declaraciones; reglas de alcance simple](#): en muchos lenguajes los nombres son globales o locales por defecto.
- [Tipado dinámico flexible](#): se chequea el tipo antes de su uso.
- [Acceso simple a las facilidades del sistema](#): proveen una manera de pedir al sistema operativo la ejecución de otro programa o realizar una operación directamente.
- [Pattern-matching sofisticado y manipulación de strings](#): uso directo de expresiones regulares extendidas.
- [Tipos de datos de alto nivel](#): permiten la funciones de tipos de alto nivel con la sintaxis y semántica del mismo lenguaje (por ejemplo contar con un arreglo indexado por caracteres, con la implementación subyacente de la tabla hash).

¿Preguntas?
